Runbook: operating and troubleshooting ragcap

For whoever runs the system day to day. Assumes the SOP has been completed. All commands run from the repository root (the folder created by git clone https://github.com/shreenathacc22/ragcap).

1. Components and where they live

Component	Where	Notes
Postgres + pgvector	Docker container ragcap-pg, volume ragcap_pgdata, 127.0.0.1:5433	restart policy: unless-stopped; needs Docker Desktop running
Credentials	the repository's .env (mode 600, gitignored)	never commit; never paste into chat or docs
LLM and embeddings	Ollama on localhost:11434	models llama3.2:3b, nomic-embed-text; override with LLM_MODEL / EMBED_MODEL / LLM_BASE_URL / LLM_API_KEY in .env
Code	ragcap/ (config, db, llm, ingest, graph, tracing), api.py, ui.py, eval.py, b1_check.py, b1_eval.py, recreate.sh	git: first commit is the golden freeze, one commit per step
Traces	traces.jsonl (gitignored, grows with every question)	one line per question
Eval reports	results/: eval_*, llama3b_*, answer_*, version_filter_*, *_summary.md	baseline eval_20261018_160824; series llama3b_summary.md; filter ablation_summary.md

2. Start, stop, health

```
open -a Docker                                  # wait until: docker info succeeds
docker start ragcap-pg                          # no-op if already running
docker exec ragcap-pg pg_isready -U ragcap -d ragcap
curl -s localhost:11434/api/tags | head -c 80    # Ollama up?
.venv/bin/uvicorn api:app --port 8000 &          # then: curl -s localhost:8000/health
docker stop ragcap-pg                           # stop (data kept in the volume)
```

/health returns document count, chunk count and the list of uploaded tables. Expected after the SOP (step 20 corpus): 16 documents, 16 chunks, [broker_inventory].

2b. After a reboot or when things 'just stopped working'

Neither Docker Desktop nor Ollama starts automatically after a reboot on this Mac, and both were found stopped once mid-series. Check them first, in this order:

```
open -a Docker && until docker info >/dev/null 2>&1; do sleep 5; done   # 30-90 s
docker start ragcap-pg && sleep 5 && docker exec ragcap-pg pg_isready -U ragcap -d ragcap
curl -sf localhost:11434/api/tags >/dev/null && echo ollama up || open -a Ollama
# run_series.sh, run_answer_ablation.sh and recreate.sh refuse to start if either service is down
```

Root cause found 2026-10-19 in the system log (/usr/bin/log show --predicate 'process == "kernel" AND eventMessage CONTAINS "System Sleep"'): every 'outage' was the Mac going to sleep. Sleep drops Docker's VM and Ollama's TCP connections; Ollama recovers on wake, Docker Desktop often does not. Fix: run_series.sh and run_answer_ablation.sh re-exec themselves under caffeinate -i, which blocks idle sleep for exactly the command's duration. For any other long command: caffeinate -i . Memory was not the cause (64% free).

3. Routine tasks

Task	Command
Add documents	.venv/bin/python -m ragcap.ingest <file-or-folder> [--version N] (re-ingesting the same name+version replaces it)

Task	Command
Add a CSV/Excel table	.venv/bin/python -m ragcap.ingest <file.csv> (table name = sanitised file name; Excel: one table per sheet)
Ask a question from the shell	.venv/bin/python try.py "question"
Version filter	ON by default since step 20g. To see the trap for a demo: VERSION_AWARE=0 .venv/bin/python try.py "q". Never set 0 in .env for normal use
Add a new version of a document	give the file front-matter (doc, version, effective_date, superseded) and ingest; then mark the old row: UPDATE documents SET superseded=true WHERE name=... AND version<N
Check the versioned corpus is sound	.venv/bin/python b1_check.py --no-trap (PASS/FAIL; run after every ingest of a versioned document)
Re-prove the filter after a change	caffeinate -i .venv/bin/python b1_eval.py --latency 3 (OFF/ON table in results/ablation_summary.md)
Try a different grader (experiment)	GRADE_MODEL=<model> GRADE_MODE=per_passage|single|escalate GRADE_MAX_TOKENS=<n> .venv/bin/python try.py "q" (defaults = production behaviour)
Run the evaluation (one file)	.venv/bin/python eval.py [--golden golden/golden_heldout.jsonl] [--tag name] (15-25 min)
Run the full series (3x held-out + 2x original)	nohup ./run_series.sh <label> <model> > results/series_<label>.out 2>&1 & (~1.5 h; pre-flight checks both services)
Inspect recent traces	tail -n 3 traces.jsonl | python -m json.tool
List what is stored	docker exec ragcap-pg psql -U ragcap -d ragcap -c "select d.name,d.version,count(c.id) from documents d join chunks c on c.doc_id=d.id group by 1,2"

4. Backup, restore, reset

```
# backup (do this before re-ingesting or changing the embedding model)
docker exec ragcap-pg pg_dump -U ragcap -d ragcap -Fc > ragcap_$(date +%Y%m%d).dump
# restore into a clean database
docker exec -i ragcap-pg pg_restore -U ragcap -d ragcap --clean --if-exists < ragcap_YYYYMMDD.dump
# full reset (DESTROYS all stored vectors and tables; re-run SOP steps 1b and 2 afterwards)
docker rm -f ragcap-pg && docker volume rm ragcap_pgdata
```

Dump files can contain document text; keep them out of git. The reset commands are destructive: check the volume name first with docker volume ls.

5. Troubleshooting

Symptom	Likely cause	Fix
Cannot connect to the Docker daemon	Docker Desktop not running	open -a Docker; wait for docker info
psycopg OperationalError / connection refused on 5433	container stopped, or Docker not started	docker start ragcap-pg; docker ps
password authentication failed	.env password differs from the one the volume was created with	The Postgres password is fixed on first volume creation. Restore the original .env, or reset the volume (section 4)
AssertionError: EMBED_DIM does not match	embedding model changed	Change EMBED_DIM and vector(N) in schema.sql, recreate chunks table, re-ingest everything
Connection refused to localhost:11434	Ollama not running	open the Ollama app or run: ollama serve
Everything answers 'not in the knowledge base'	grader returned [] (3B model), or nothing ingested	check /health counts; run the pipeline nodes by hand (see section 7); try a larger LLM_MODEL

Symptom	Likely cause	Fix
Document question answered with database numbers	router sent it to database	Fixed in step 9 (router now sees table columns + guard). If it returns, run the router probe in Runbook section 7 and add an unrelated example to the router prompt
Rewritten question contains words the user never said	router prompt leaked wording into 'standalone'	Fixed in step 13: standalone must be verbatim. Check s['standalone'] in the stage probe
Right passage retrieved but 'not in the knowledge base'	grader dropped it (strict on hostnames/ports)	Probe grade() per passage; loosen the grade prompt or try a larger model; check relevant list vs chunks
Same question gives different SQL on different runs	3B model variability, prompt rules interact	Read the 'sql' field in traces.jsonl; keep rules short; never add rules that mention WHERE/LIMIT without 'ONLY if'; re-run eval 3x
Eval shows 'amended doc-008'	expected: amendment file applied	See golden/amendments.jsonl for the reason; never edit golden.jsonl
Eval log ends with JSONDecodeError	old parse_json (before commit 9bbd8c4) on truncated model JSON	git pull / check graph.py has the never-raising parse_json; raise max_tokens of the failing node
Eval log: 'Connection refused' on port 11434 or 5433	Ollama or Postgres stopped during the run	Section 2b; re-run the whole series (partial logs are not comparable)
Eval metrics show errors > 0	one or more cases hit an exception and were recorded as route 'error'	grep ERROR results/<run>.log; fix the cause; re-run before trusting the percentages
results/<label>_series.done missing after 2 h	a run crashed or the Mac slept	pgrep -fl run_series; tail results/<label>_*.log; check Energy Saver / keep the lid open during a series
Connection refused to 5433 or 11434 in the middle of a run that was fine	the Mac went to sleep (check: /usr/bin/log show --last 3h --predicate 'eventMessage CONTAINS "System Sleep"')	open -a Docker; docker start ragcap-pg; re-run the whole series under caffeinate -i (section 2b). Partial logs are not comparable
b1_check.py FAIL: 'need exactly one current and one superseded version'	a new version was ingested without marking the old one superseded	UPDATE documents SET superseded=true WHERE name='<title>' AND version<<new>; re-run b1_check.py
Answers quote a retired value (old port, old threshold)	VERSION_AWARE is off, or the old document's row is not marked superseded	check docker exec ragcap-pg psql ... -c "select name,version,superseded from documents"; set VERSION_AWARE=1; b1_eval.py shows superseded rate
Everything slow, load average > cores, a llama-server at 500%+ CPU for many minutes	a killed client left Ollama generating; or an 8B model given a long prompt on this CPU	1) find the client: pgrep -fl 'venv/bin/python' and kill it by PID (env vars are not in the command line, so pkill -f on VAR=... misses the Python child); 2) unload the model through the API: curl localhost:11434/api/generate -d '{"model":"qwen3:8b","keep_alive":0}' (the server path is a hash, grepping for the model name finds nothing); 3) probe with try.py 'Thanks!'. Note chat() retries on connection errors, so an orphaned client re-sends its request when Ollama comes back. Do not run qwen3:8b multi-passage prompts on this Mac (SOP step 19)
SQL answer has wrong numbers but SQL is right	generate_sql paraphrased badly	Fixed in step 10 (copy numbers exactly). If it returns, compare the Rows in the trace with the answer text

Symptom	Likely cause	Fix
'only a single read-only SELECT is allowed'	validator rejected generated SQL	expected safety behaviour; the agent retries once with the error
First request after idle takes 30+ s	Ollama loading the model into memory	send a warm-up request; keep models loaded (OLLAMA_KEEP_ALIVE)
pip install docling then ModuleNotFoundError / numpy / torch errors	see section 6	do not install Docling in the .venv

6. Known limitation: Docling on this Mac

Docling 2.140 installs on this Intel Mac but cannot run. The last PyTorch wheel for Intel macOS is 2.2.2; current transformers requires torch 2.4 or newer, and the Docling layout model (rt_detr_v2) needs a newer transformers plus numpy 2, while torch 2.2.2 wheels need numpy below 2. Pinning numpy<2 and transformers<4.50 moves the failure to the layout model load ('numpy has no attribute long'). The test venv was deleted. Options: run Docling on an Apple Silicon or Linux machine and ingest the resulting Markdown; use a different parser (pypdf is the current fallback and loses tables/layout); or run Docling in a Linux container.

7. Debugging a wrong answer, stage by stage

```
.venv/bin/python - <<'EOF'
import ragcap.graph as g
s = g.router({"question": "YOUR QUESTION", "history": [], "trace": []})
print(s["route"], "|", s["standalone"])           # 1. did it route correctly, was the question rewritten sensibly?
r = g.retrieve({**s}); print([(c["doc"], c["dist"]) for c in r["chunks"]])   # 2. is the right doc in the top 6?
gr = g.grade({**s, **r, "trace": []}); print([c["doc"] for c in gr["relevant"]])  # 3. did grade keep it?
EOF
```

The stage that first goes wrong is the one to fix. Retrieval problems show as a high cosine distance or the wrong document; grading problems as an empty relevant list while the right document was retrieved.

8. Security notes

- The database port is bound to 127.0.0.1 only. - The SQL agent can only run one SELECT/WITH statement, in a READ ONLY transaction with a 5 s timeout, but it runs as the same database user that owns the tables; for anything beyond a local demo give it a separate read-only role. - /upload writes files into uploads/ (gitignored) and has no authentication or size limit; do not expose the API beyond localhost. - Uploaded text is passed to the model unfiltered, so a malicious document can attempt prompt injection.

9. Evaluation pass/fail guide

Metric	Run 1 baseline	Run 2	Run 3 (current code)	Accept if
Router accuracy	88.9%	100%	100%	>= 95%
Answer correct	77.8%	92.6%	88.9%	>= 90% (mean of 3 runs)
Retrieval hit rate (document)	81.3%	100%	100%	>= 90%
Unanswerable refusals	2/2	2/2	2/2	2/2 (do not regress)
SQL cases	4/6	6/6	5/6	6/6

Run-to-run variability: with temperature 0 the local 3B model still changes which cases fail between runs because the router's rewritten question and the grader's choices differ slightly. Judge a change on the mean of at least 3 runs, and look at which case ids fail, not only the percentage.

A change is accepted only if it improves or holds every row above on the same frozen golden set. Never edit golden/golden.jsonl to make a run pass; add new cases in a new file and commit them separately.

10. Rebuilding elsewhere

docs/RECREATE.md (PDF: RECREATE_ragcap.pdf) is the from-zero guide with a scenario catalogue and the fingerprints; recreate.sh runs it unattended under caffeinate and exits 1 on any fingerprint mismatch. Run it after any change to the corpus, the schema or retrieve() to prove nothing drifted.

11. Change-control procedure for prompt or pipeline changes

#	Action	Command / rule
1	Reproduce the failure on one question	.venv/bin/python try.py "question" ; then the stage probe (section 7)
2	Identify the first stage that goes wrong	router -> retrieve -> grade -> generate (or sql_agent -> generate_sql)
3	Change ONE prompt or rule	edit ragcap/graph.py; never paste golden questions into prompts as examples (leakage)
4	Probe the failing question plus 3 neighbours	.venv/bin/python try.py ... (watch for the fix leaking into other queries)
5	Commit before measuring	git add -A && git commit -m "what and why"
6	Full evaluation in background	nohup .venv/bin/python eval.py > results/eval_runN.log 2>&1 &
7	Compare to the previous run by failing ids	grep BAD results/eval_runN.log ; sed -n '/## Metrics/,$p' results/eval_runN.log
8	Record the run in the SOP results table	python docs/build_pdfs.py (needs reportlab) and commit